it’s still linear, O(|V|+|E|), but it has the great advantage of avoid any requirement for specific data structures to store the candidate embeddings. This algorithm is currently one of the state-of-the-art solutions to find planar embeddings for (planar) graphs; the best part is that you can find an implementation here:

https://www.boost.org/doc/libs/1_67_0/libs/graph/doc/boyer_myrvold.html.

The last algorithm we are going to mention is the planarity testing algorithm[22] by Fraysseix, de Mendes and Rosenstiehl, which is the other state-of-the-art algorithm in this area. It characterizes planar graphs in terms of a left-right ordering of the edges in a depth-first search tree, and its implementation is DFS-based, although obviously the DFS method needs to be modified accordingly and it uses *Trémaux trees*, special spanning trees produced by a DFS visit of a graph.

# 15.3 Non-Planar Graphs

Now that we have at least one planarity testing algorithm in our tool-belt, we can look at our task, visualizing graphs nicely on screen, with more confidence. Some of the planarity testing algorithms also output a planar embedding, and that gives us a good starting point.

And yet, we have a long way to go.

Let’s start with two considerations: